IMDB Sentiment-Analyse: Naive Bayes vs. LSTM

Dieses Notebook lädt das IMDB-Dataset aus dem Ordner `Data/` und trainiert zwei Klassifikatoren: einen klassischen Ansatz (TF-IDF + Naive Bayes) und ein Deep-Learning-Modell (LSTM). Zum Schluss vergleichen wir Metriken, Laufzeit und Interpretierbarkeit und geben Empfehlungen.

Hinweis: Die Zellen sind so gestaltet, dass das Notebook auf einem typischen Studenten-Laptop lauffähig ist (begrenzte Vokabulargröße, wenige Epochen).

In [11]:
```
# Section 1: Imports und Einstellungen
import os
import random
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn import metrics
import joblib
import nltk
from nltk.corpus import stopwords
import re
# TensorFlow / Keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Bidirectional, Dropo
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Reproduzierbarkeit
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

# Zeige Versionen
print('pandas', pd.__version__)
print('numpy', np.__version__)
print('tensorflow', tf.__version__)
```

```
pandas 2.3.3
numpy 2.3.5
tensorflow 2.20.0
```

In [12]:
```
# Section 2: Daten laden (robustes Snippet)
data_path = os.path.join('Data', 'IMDB Dataset.csv')
```

```
if not os.path.exists(data_path):
    raise FileNotFoundError(f'Datei nicht gefunden: {data_path} - stelle sicher,

df = pd.read_csv(data_path)
print('Shape:', df.shape)
display(df.head(5))
display(df.info())
```

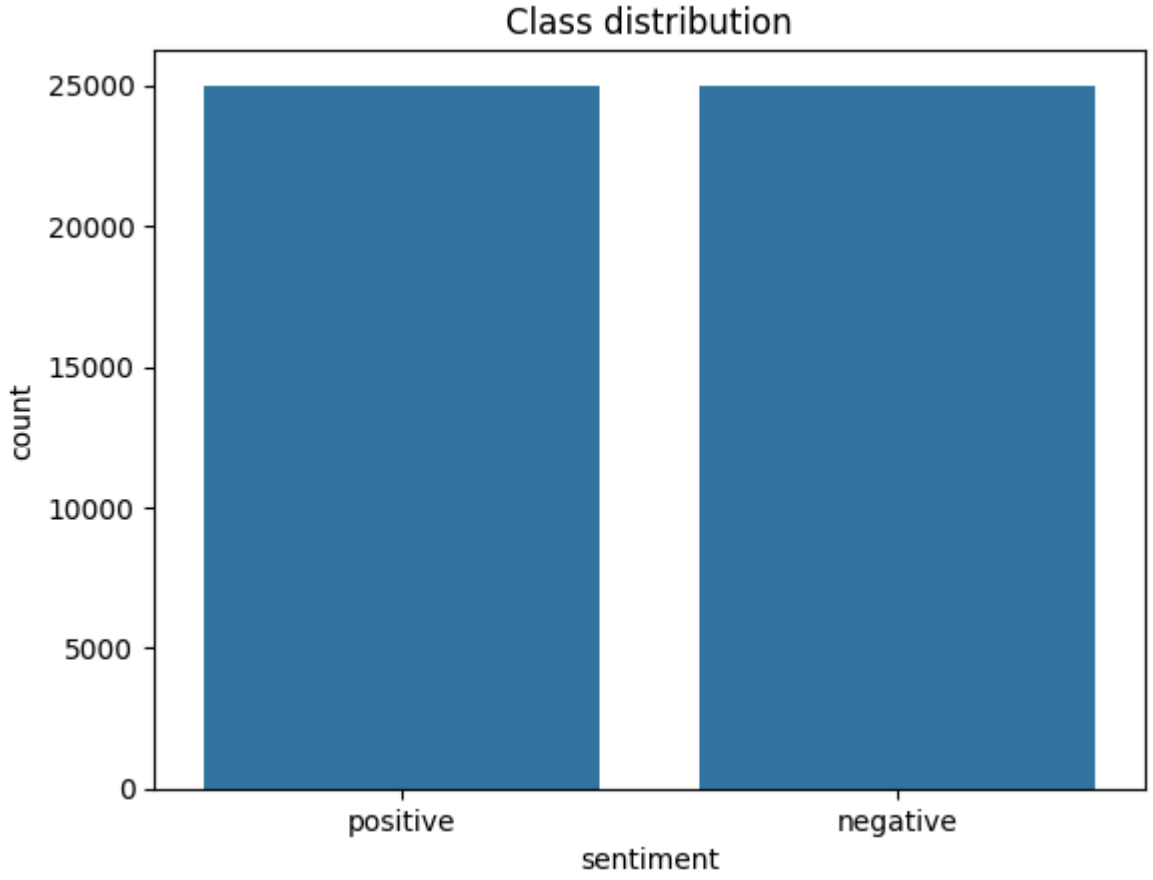
```
Shape: (50000, 2)
```

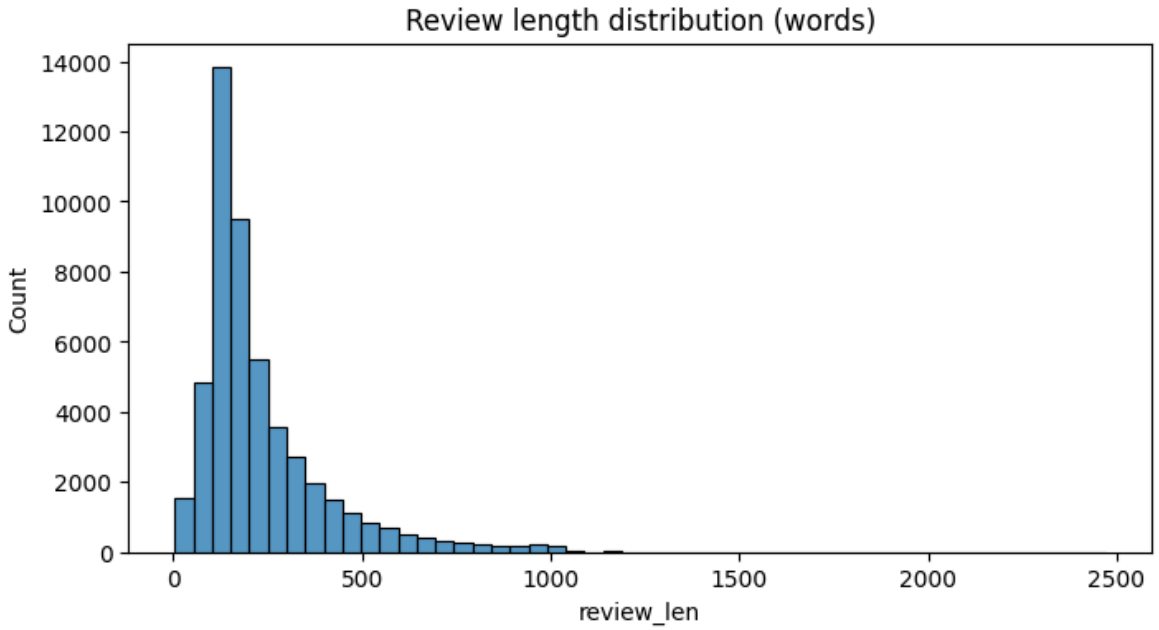
	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 2 columns):
 #   Column     Non-Null Count  Dtype 
---  ------     --------------  ----- 
 0   review     50000 non-null  object
 1   sentiment  50000 non-null  object
dtypes: object(2)
memory usage: 781.4+ KB
None
```

In [13]:
```
# Section 3: EDA - Klassenverteilung & Längenverteilung
if 'sentiment' in df.columns and 'review' in df.columns:
    display(df['sentiment'].value_counts())
    sns.countplot(data=df, x='sentiment')
    plt.title('Class distribution')
    plt.show()
    df['review_len'] = df['review'].apply(lambda x: len(str(x).split()))
    plt.figure(figsize=(8,4))
    sns.histplot(df['review_len'], bins=50)
    plt.title('Review length distribution (words)')
    plt.show()
else:
    print('Erwarte Spalten `review` und `sentiment` im Dataset')
```

```
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```





In [14]:
```python
# Section 4: Preprocessing - clean_text Funktion
nltk.download('stopwords')
STOPWORDS = set(stopwords.words('english'))

def clean_text(text, remove_stopwords=True):
    if not isinstance(text, str):
        text = str(text)
    # HTML entfernen
    text = re.sub(r'<.*?>', ' ', text)
    # non-letters entfernen
    text = re.sub(r'[^a-zA-Z]', ' ', text)
    text = text.lower()
    tokens = text.split()
```

```
    if remove_stopwords:
        tokens = [t for t in tokens if t not in STOPWORDS]
    return ' '.join(tokens)

# Sample Anwendung (erst nur auf kleinen Subset um Zeit zu sparen)
df['clean_review'] = df['review'].astype(str).str[:500].apply(clean_text)  # saf
display(df[['review','clean_review']].head())
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Stefan\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

	review	clean_review
0	One of the other reviewers has mentioned that ...	one reviewers mentioned watching oz episode ho...
1	A wonderful little production. The...	wonderful little production filming technique ...
2	I thought this was a wonderful way to spend ti...	thought wonderful way spend time hot summer we...
3	Basically there's a family where a little boy ...	basically family little boy jake thinks zombie...
4	Petter Mattei's "Love in the Time of Money" is...	petter mattei love time money visually stunnin...

In [15]:
```
# Label-Encoding und train/test split (Section 4/6)
label_map = {'positive':1, 'negative':0}
df['label'] = df['sentiment'].map(label_map)
X = df['clean_review']
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratif
print('Train/Test sizes:', X_train.shape, X_test.shape)
```

```
Train/Test sizes: (40000,) (10000,)
```

Modell 1: TF-IDF + Naive Bayes (klassischer Ansatz)

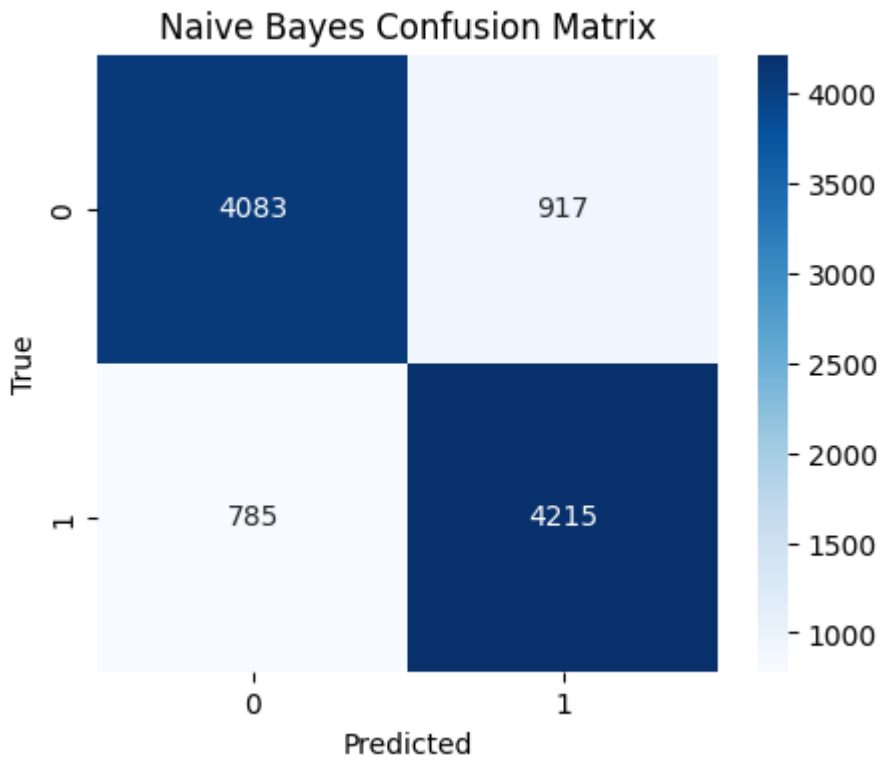
Wir verwenden `TfidfVectorizer` + `MultinomialNB`. Dieser Ansatz ist schnell, gut interpretierbar (Top-Features) und für Bag-of-Words-Textklassifikation häufig sehr effektiv.

In [16]:
```
# Utility: Metriken-Funktion
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_sc

def compute_metrics(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred)
    rec = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    return {'accuracy': acc, 'precision': prec, 'recall': rec, 'f1': f1}

def plot_confusion(y_true, y_pred, labels=[0,1], title='Confusion Matrix'):
    cm = confusion_matrix(y_true, y_pred, labels=labels)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
```

```
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.title(title)
    plt.show()
```

In [17]:
```
# Section 6: Train Naive Bayes
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

nb_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        max_features=50000,        # bigger vocab
        ngram_range=(1, 2),        # keep bigrams
        min_df=2,                  # ignore super-rare words
        max_df=0.95,               # ignore extremely common tokens
        sublinear_tf=True,         # log-scaling of term frequency
        stop_words='english'
    )),
    ('nb', MultinomialNB(alpha=0.5))  # slightly stronger smoothing
])

t0 = time.time()
nb_pipeline.fit(X_train, y_train)
t1 = time.time()
print(f'Training Naive Bayes took {t1-t0:.2f} sec')
y_pred_nb = nb_pipeline.predict(X_test)
metrics_nb = compute_metrics(y_test, y_pred_nb)
print('Naive Bayes results:', metrics_nb)
print('Classification report:', classification_report(y_test, y_pred_nb))
plot_confusion(y_test, y_pred_nb, title='Naive Bayes Confusion Matrix')
# Speichere Modell
joblib.dump(nb_pipeline, 'nb_pipeline.joblib')
print('NB pipeline saved to nb_pipeline.joblib')
```

```
Training Naive Bayes took 3.89 sec
Naive Bayes results: {'accuracy': 0.8298, 'precision': 0.8213172252533125, 'recal
l': 0.843, 'f1': 0.8320173707066719}
Classification report:               precision    recall  f1-score   support

           0       0.84      0.82      0.83      5000
           1       0.82      0.84      0.83      5000

    accuracy                           0.83     10000
   macro avg       0.83      0.83      0.83     10000
weighted avg       0.83      0.83      0.83     10000

Naive Bayes results: {'accuracy': 0.8298, 'precision': 0.8213172252533125, 'recal
l': 0.843, 'f1': 0.8320173707066719}
Classification report:               precision    recall  f1-score   support

           0       0.84      0.82      0.83      5000
           1       0.82      0.84      0.83      5000

    accuracy                           0.83     10000
   macro avg       0.83      0.83      0.83     10000
weighted avg       0.83      0.83      0.83     10000
```




```
NB pipeline saved to nb_pipeline.joblib
```

Modell 2: LSTM (Deep Learning)

Wir erstellen ein einfaches LSTM-Modell mit Embedding-Layer. Längere Trainingszeit und höhere Rechenkosten können auftreten; deshalb begrenzen wir Vokabulargröße und Epochen.

BiLSTM + GloVe -> Glove uses data from wikipedia and is not focused on Movie reviews

In [ ]:
```python
# Section 8: BiLSTM + GloVe Embeddings on IMDB

import time
import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers
import joblib

# Tokenisierung & Padding

MAX_WORDS = 50000   # Vokabulargröße
MAX_LEN   = 300
EMBED_DIM = 100

# Sicherstellen, dass X_train / X_test Strings sind
X_train_text = X_train.astype(str)
X_test_text  = X_test.astype(str)

# Tokenizer nur auf Trainingsdaten fitten
tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train_text)
```

```
# Texte -> Integer-Sequenzen
X_train_seq = tokenizer.texts_to_sequences(X_train_text)
X_test_seq  = tokenizer.texts_to_sequences(X_test_text)

print("Example training review:\n", X_train_text.iloc[0][:300])
print("Example training sequence (first 30 tokens):", X_train_seq[0][:30])

# Padding auf gleiche Länge
X_train_pad = pad_sequences(
    X_train_seq, maxlen=MAX_LEN,
    padding='post', truncating='post'
)
X_test_pad = pad_sequences(
    X_test_seq, maxlen=MAX_LEN,
    padding='post', truncating='post'
)

print("Train tensor shape:", X_train_pad.shape)
print("Test tensor shape:", X_test_pad.shape)

# Labels in numpy-Arrays (float32) umwandeln
y_train_arr = np.asarray(y_train).astype("float32")
y_test_arr  = np.asarray(y_test).astype("float32")

print("y_train class distribution:", np.bincount(y_train_arr.astype("int32")))
print("y_test class distribution:", np.bincount(y_test_arr.astype("int32")))

# ------------------------------------------------
# 2) GloVe-Embeddings laden & Embedding-Matrix bauen
# ------------------------------------------------
# Pfad zur GloVe-Datei (passe an, falls sie woanders liegt)
GLOVE_PATH = "glove.6B.100d.txt"

if not os.path.exists(GLOVE_PATH):
    raise FileNotFoundError(
        f"{GLOVE_PATH} not found. "
        "Download GloVe 6B (100d) and place glove.6B.100d.txt in the working dir
    )

print("Loading GloVe embeddings from", GLOVE_PATH)

embeddings_index = {}
with open(GLOVE_PATH, "r", encoding="utf8") as f:
    for line in f:
        values = line.strip().split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype="float32")
        embeddings_index[word] = coefs

print("Loaded word vectors:", len(embeddings_index))

word_index = tokenizer.word_index
num_words = min(MAX_WORDS, len(word_index) + 1)

embedding_matrix = np.zeros((num_words, EMBED_DIM), dtype="float32")
missing = 0

for word, i in word_index.items():
    if i >= MAX_WORDS:
        continue
```

```
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None:
        embedding_matrix[i] = embedding_vector
    else:
        # Wort nicht im GloVe-Vokabular
        missing += 1

print(f"Embedding matrix shape: {embedding_matrix.shape}")
print(f"Words without GloVe vector: {missing}")

# Embedding-Layer mit GloVe initialisieren
embedding_layer = layers.Embedding(
    input_dim=num_words,
    output_dim=EMBED_DIM,
    weights=[embedding_matrix],
    input_length=MAX_LEN,
    trainable=True  # erst einfrieren, später ggf. True setzen für Fine-Tuning
)

# ------------------------------------------------
# 3) BiLSTM-Modell definieren (mit GloVe)
# ------------------------------------------------
inputs = tf.keras.Input(shape=(MAX_LEN,), dtype="int32")
x = embedding_layer(inputs)

x = layers.Bidirectional(
    layers.LSTM(128, return_sequences=True)
)(x)
x = layers.Dropout(0.3)(x)

x = layers.Bidirectional(
    layers.LSTM(64)
)(x)

x = layers.Dense(64)(x)
x = layers.LeakyReLU(alpha=0.1)(x)
x = layers.Dropout(0.4)(x)

outputs = layers.Dense(1, activation="sigmoid")(x)

model = tf.keras.Model(inputs=inputs, outputs=outputs)

model.compile(
    loss="binary_crossentropy",
    optimizer=tf.keras.optimizers.Adam(learning_rate=2e-4),
    metrics=["accuracy"]
)

model.summary()

# ------------------------------------------------
# 4) Training mit EarlyStopping
# ------------------------------------------------
batch_size = 64
epochs = 10

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
```

```
)

t0 = time.time()
history = model.fit(
    X_train_pad, y_train_arr,
    batch_size=batch_size,
    epochs=epochs,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
t1 = time.time()
print(f"Training BiLSTM + GloVe took {t1 - t0:.2f} sec")

# ------------------------------------------------
# 5) Evaluation auf dem Testset
# ------------------------------------------------
t0 = time.time()
y_pred_proba = model.predict(X_test_pad, batch_size=256)
t1 = time.time()
print(f"Prediction on test set took {t1 - t0:.2f} sec")

print("Predicted probabilities: min =", float(y_pred_proba.min()),
      "max =", float(y_pred_proba.max()),
      "mean =", float(y_pred_proba.mean()))

# Wahrscheinlichkeiten -> Klassen (0/1)
y_pred_bilstm_glove = (y_pred_proba >= 0.5).astype("int32").ravel()

print("y_pred_bilstm_glove class distribution:",
      np.bincount(y_pred_bilstm_glove))

# ------------------------------------------------
# 6) Metriken & Confusion Matrix
# ------------------------------------------------
metrics_bilstm_glove = compute_metrics(y_test_arr, y_pred_bilstm_glove)
print("BiLSTM + GloVe results:", metrics_bilstm_glove)
print("Classification report:\n",
      classification_report(y_test_arr, y_pred_bilstm_glove))

plot_confusion(y_test_arr, y_pred_bilstm_glove,
               title="BiLSTM + GloVe Confusion Matrix")

# ------------------------------------------------
# 7) Modell & Tokenizer speichern
# ------------------------------------------------
model.save("bilstm_glove_imdb_model.h5")
joblib.dump(tokenizer, "tokenizer_bilstm_glove.joblib")
print("BiLSTM + GloVe model saved to bilstm_glove_imdb_model.h5")
print("Tokenizer saved to tokenizer_bilstm_glove.joblib")

# Optional: falls ihr eine 'results'-Liste habt:
# results.append({
#     'model': 'BiLSTM + GloVe',
#     **metrics_bilstm_glove
# })
```

```
Example training review:
 caught little gem totally accident back revival theatre see two old silly sci fi
movies theatre packed full warning showed bunch sci fi short spoofs get us mood s
omewhat amusing came within seconds audience hysterics biggest laugh came showed
princess laia huge cinnamon buns instead hair head looks
Example training sequence (first 30 tokens): [707, 42, 1075, 307, 1248, 56, 6346,
1131, 11, 38, 55, 507, 557, 471, 17, 1131, 2149, 243, 1417, 897, 552, 557, 471, 2
03, 6202, 22, 100, 1208, 563, 1020]
Train tensor shape: (40000, 300)
Test tensor shape: (10000, 300)
y_train class distribution: [20000 20000]
y_test class distribution: [5000 5000]
Loading GloVe embeddings from glove.6B.100d.txt
Loaded word vectors: 400000
Embedding matrix shape: (50000, 100)
Words without GloVe vector: 6952
```

```
f:\FHTechnikum\Semester 5\NLP\venv\Lib\site-packages\keras\src\layers\core\embedd
ing.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
f:\FHTechnikum\Semester 5\NLP\venv\Lib\site-packages\keras\src\layers\activations
\leaky_relu.py:41: UserWarning: Argument `alpha` is deprecated. Use `negative_slo
pe` instead.
  warnings.warn(
```

Model: "functional_1"

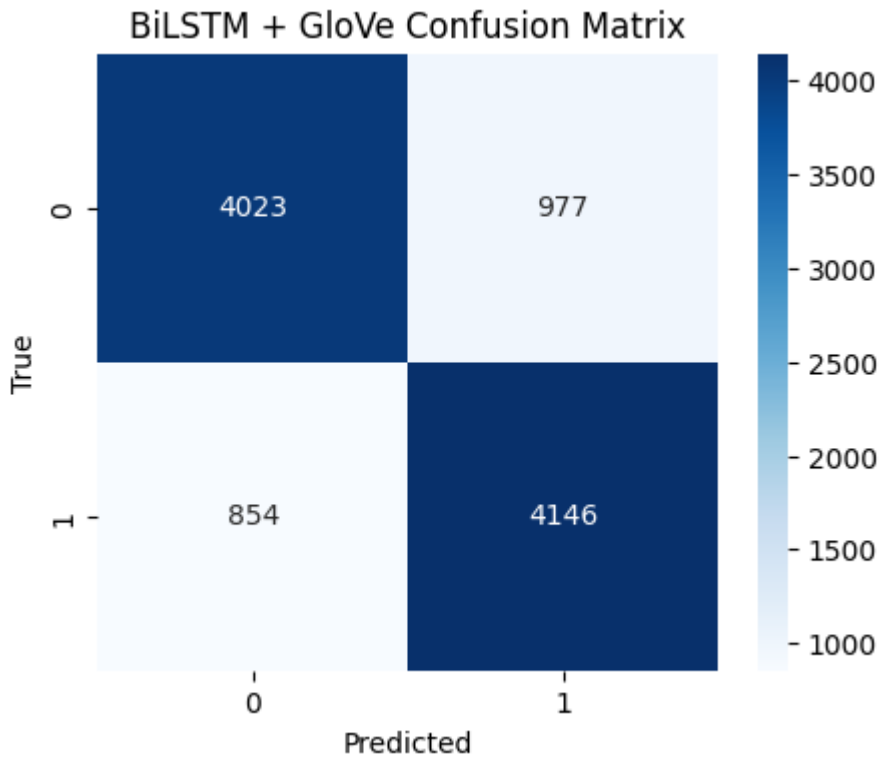
Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 300)	0
embedding_1 (Embedding)	(None, 300, 100)	5,000,000
bidirectional_2 (Bidirectional)	(None, 300, 256)	234,496
dropout_2 (Dropout)	(None, 300, 256)	0
bidirectional_3 (Bidirectional)	(None, 128)	164,352
dense_2 (Dense)	(None, 64)	8,256
leaky_re_lu_1 (LeakyReLU)	(None, 64)	0
dropout_3 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 1)	65

Total params: 5,407,169 (20.63 MB)

Trainable params: 5,407,169 (20.63 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 232s 460ms/step - accuracy: 0.7275 - loss: 0.5400 -
val_accuracy: 0.7797 - val_loss: 0.4637
Epoch 2/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 250s 501ms/step - accuracy: 0.8030 - loss: 0.4349 -
val_accuracy: 0.7983 - val_loss: 0.4212
Epoch 3/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 261s 522ms/step - accuracy: 0.8366 - loss: 0.3722 -
val_accuracy: 0.8165 - val_loss: 0.3989
Epoch 4/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 261s 523ms/step - accuracy: 0.8663 - loss: 0.3188 -
val_accuracy: 0.8215 - val_loss: 0.4032
Epoch 5/10
500/500 ━━━━━━━━━━━━━━━━━━━━ 261s 522ms/step - accuracy: 0.8942 - loss: 0.2689 -
val_accuracy: 0.8219 - val_loss: 0.4485
Training BiLSTM + GloVe took 1265.24 sec
40/40 ━━━━━━━━━━━━━━━━━━━━ 15s 373ms/step
Prediction on test set took 15.19 sec
Predicted probabilities: min = 0.008283955976366997 max = 0.9918140172958374 mean
= 0.49866485595703125
y_pred_bilstm_glove class distribution: [4877 5123]
BiLSTM + GloVe results: {'accuracy': 0.8169, 'precision': 0.8092914308022643, 're
call': 0.8292, 'f1': 0.8191247653857552}
Classification report:
              precision    recall  f1-score   support

         0.0       0.82      0.80      0.81      5000
         1.0       0.81      0.83      0.82      5000

    accuracy                           0.82     10000
   macro avg       0.82      0.82      0.82     10000
weighted avg       0.82      0.82      0.82     10000
```



```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
BiLSTM + GloVe model saved to bilstm_glove_imdb_model.h5
Tokenizer saved to tokenizer_bilstm_glove.joblib
```

LSTM (andom Embeddings)

In [19]:
```
# Section 7: Tokenisierung und Sequenzvorbereitung
MAX_NUM_WORDS = 20000
MAX_LEN = 200
tokenizer = Tokenizer(num_words=MAX_NUM_WORDS, oov_token='<OOV>')
tokenizer.fit_on_texts(X_train)
X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)
X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='post', truncat
X_test_pad = pad_sequences(X_test_seq, maxlen=MAX_LEN, padding='post', truncatin
print('Vocabulary size (approx):', min(MAX_NUM_WORDS, len(tokenizer.word_index))
print('X_train_pad shape:', X_train_pad.shape)
```

```
Vocabulary size (approx): 20000
X_train_pad shape: (40000, 200)
```

In [ ]:
```
# Section 8: LSTM Modellaufbau und Training
EMBEDDING_DIM = 100
model = Sequential([
    Embedding(input_dim=MAX_NUM_WORDS, output_dim=EMBEDDING_DIM, input_length=MA
    Bidirectional(LSTM(64, dropout=0.2, recurrent_dropout=0.2)),
    Dense(32, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy']
model.summary()
# Callbacks
es = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
# train (keeping epochs small for student machines)
t0 = time.time()
history = model.fit(X_train_pad, y_train, epochs=5, batch_size=128,
                    validation_split=0.1, callbacks=[es], verbose=1)
t1 = time.time()
print(f'LSTM training took {t1-t0:.2f} sec')
# Speichere Modell und Tokenizer
model.save('lstm_model.h5')
joblib.dump(tokenizer, 'tokenizer.joblib')
print('LSTM model and tokenizer saved')
```

```
f:\FHTechnikum\Semester 5\NLP\venv\Lib\site-packages\keras\src\layers\core\embedd
ing.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
bidirectional_4 (Bidirectional)	?	0 (unbuilt)
dense_4 (Dense)	?	0 (unbuilt)
dropout_4 (Dropout)	?	0
dense_5 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

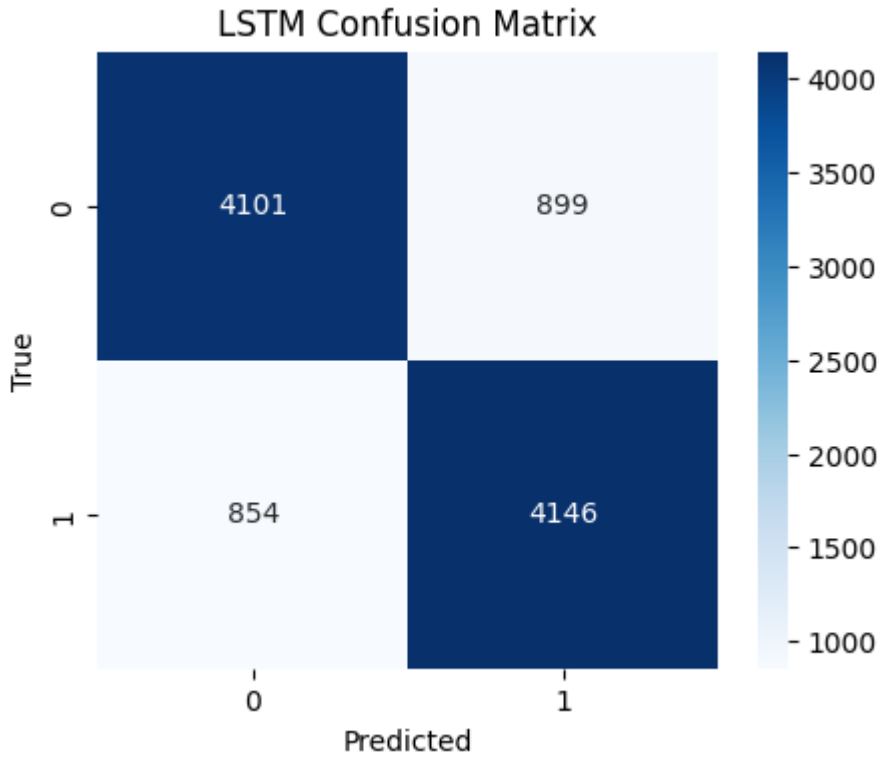
Trainable params: 0 (0.00 B)

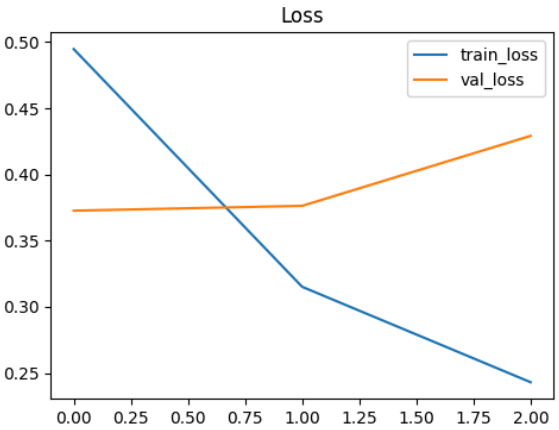
Non-trainable params: 0 (0.00 B)

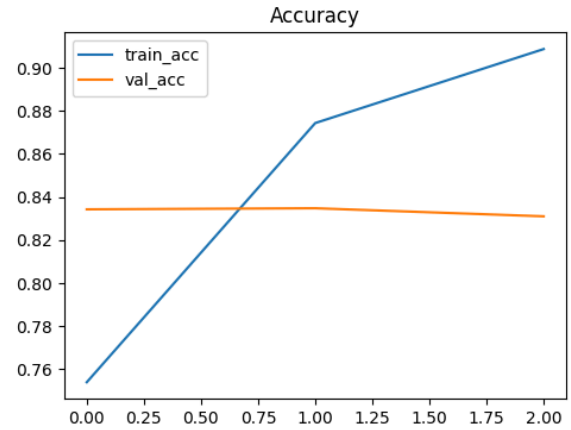
```
Epoch 1/5
282/282 ━━━━━━━━━━━━━━━━━━━━ 117s 403ms/step - accuracy: 0.7540 - loss: 0.4947 -
val_accuracy: 0.8342 - val_loss: 0.3726
Epoch 2/5
282/282 ━━━━━━━━━━━━━━━━━━━━ 113s 403ms/step - accuracy: 0.8743 - loss: 0.3151 -
val_accuracy: 0.8347 - val_loss: 0.3763
Epoch 3/5
282/282 ━━━━━━━━━━━━━━━━━━━━ 113s 400ms/step - accuracy: 0.9087 - loss: 0.2430 -
val_accuracy: 0.8310 - val_loss: 0.4291
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
LSTM training took 342.79 sec
LSTM model and tokenizer saved
```

In [21]:
```
# Section 10: Evaluation LSTM
y_probs = model.predict(X_test_pad, batch_size=128)
y_pred_lstm = (y_probs.flatten() >= 0.5).astype(int)
metrics_lstm = compute_metrics(y_test, y_pred_lstm)
print('LSTM results:', metrics_lstm)
print('Classification report:', classification_report(y_test, y_pred_lstm))
plot_confusion(y_test, y_pred_lstm, title='LSTM Confusion Matrix')
# Plot Training History
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='train_loss')
plt.plot(history.history['val_loss'], label='val_loss')
plt.legend(); plt.title('Loss')
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='train_acc')
plt.plot(history.history['val_accuracy'], label='val_acc')
plt.legend(); plt.title('Accuracy')
plt.show()
```

```
79/79 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step
LSTM results: {'accuracy': 0.8247, 'precision': 0.8218037661050545, 'recall': 0.8
292, 'f1': 0.8254853160776505}
Classification report:               precision    recall  f1-score   support

           0       0.83      0.82      0.82      5000
           1       0.82      0.83      0.83      5000

    accuracy                           0.82     10000
   macro avg       0.82      0.82      0.82     10000
weighted avg       0.82      0.82      0.82     10000
```







In [22]:
```python
# Section 11: Vergleich beider Modelle in einer Tabelle
results = pd.DataFrame([
    {'model':'NaiveBayes', **metrics_nb},
    {'model':'LSTM', **metrics_lstm}
])
display(results)
results.to_csv('model_comparison.csv', index=False)
print('Vergleich gespeichert als model_comparison.csv')
```

	model	accuracy	precision	recall	f1
0	NaiveBayes	0.8298	0.821317	0.8430	0.832017
1	LSTM	0.8247	0.821804	0.8292	0.825485

```
Vergleich gespeichert als model_comparison.csv
```

Interpretation & Diskussion (kurz)

- Naive Bayes ist sehr schnell, interpretiert durch Top-Features (TF-IDF Gewichtungen).
- LSTM kann Kontext und Reihenfolge lernen, ist aber rechenaufwändiger und benötigt mehr Daten oder Regularisierung, um nicht zu überfitten.
- Wenn NB ähnliche oder bessere Metriken liefert, ist der klassische Ansatz für Produktions-Einsätze oft ausreichend; andernfalls lohnt sich Fein-Tuning oder der Einsatz vortrainierter Embeddings/Transformer.

In [23]:
```
# Section 12: Top-Features aus NB anzeigen (Interpretierbarkeit)
tfidf = nb_pipeline.named_steps['tfidf']
nb = nb_pipeline.named_steps['nb']
feature_names = tfidf.get_feature_names_out()
class0_top = np.argsort(nb.feature_log_prob_[0])[-20:]
class1_top = np.argsort(nb.feature_log_prob_[1])[-20:]
print('Top features negative:')
print(feature_names[class0_top])
print('Top features positive:')
print(feature_names[class1_top])
```

```
Top features negative:
['way' 'know' 'think' 'watch' 'characters' 'make' 'seen' 'people' 'worst'
 'story' 'time' 'movies' 'acting' 'plot' 'really' 'good' 'like' 'bad'
 'film' 'movie']
Top features positive:
['characters' 'way' 'watch' 'years' 'think' 'life' 'films' 'movies'
 'people' 'seen' 'love' 'really' 'best' 'time' 'good' 'like' 'story'
 'great' 'film' 'movie']
```

Reproduzierbarkeit & Hinzufügungen

- Modelle: `nb_pipeline.joblib`, `lstm_model.h5`, `tokenizer.joblib` wurden gespeichert.
- Paketversionen oben ausgeben; für vollständige Reproduzierbarkeit siehe `requirements.txt`.
- Für weitergehende Verbesserungen: Hyperparameter-Tuning (GridSearchCV für NB-Pipeline), pretrained embeddings (GloVe), oder Transformer-Modelle (z.B. DistilBERT).

How to run

1. Installiere Abhängigkeiten: `pip install -r requirements.txt`
2. Öffne dieses Notebook in VS Code oder Jupyter und führe Zellen nacheinander aus.
3. Falls NLTK-Stopwords nicht vorhanden sind, wird der Download in einer Zelle ausgeführt.